

Exercise 3: Full-Stack Integration, Error Handling, and Optimistic Rollback

Context: Enterprise applications require consistent, contract-driven error shapes and resilient user experience layers. When an exception occurs on the backend, it must be returned as a structured payload rather than unformatted system track traces. On the frontend, state operations must be responsive, applying optimistic updates immediately while maintaining fallback snapshot mechanisms to gracefully recover if a background database write fails.

Part A: Server-Side ProblemDetails & Angular Error Interceptor

First, we will ensure our .NET API formats all exception and validation errors as RFC 7807

ProblemDetails JSON objects, then write an Angular functional **HttpInterceptor** to catch HTTP errors globally.

1. **Enable Backend Document Mappings:** Open **Program.cs** in your Web API project. Register **ProblemDetails** middleware services:

```
// Add RFC 7807 ProblemDetails support to the DI container
builder.Services.AddProblemDetails();
```

2. **Inject Status Code Shapers:** Scroll down to the middleware pipeline (after **builder.Build()**) and enable the status code pages middleware:

```
app.UseStatusCodePages(); // Converts 4xx/5xx responses into
standard ProblemDetails payloads
```

3. **Build the Frontend Global Error Interceptor:** Create **src/app/interceptors/error.interceptor.ts** in your Angular project:

```
import { HttpInterceptorFn, HttpResponse } from
'@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);

  return next(req).pipe(
    catchError((err: HttpResponse) => {
      // Extract the .NET RFC 7807 ProblemDetails detail property
      const detailMessage = err.error?.detail ?? 'A system error
occurred. Please try again.';

      if (err.status === 401) {
```

```

        // Redirect expired or unauthenticated sessions back to
login
        router.navigate(['/login']);
    } else {
        // Surface structured error to developer console / UI
notification
        console.error('API Error Response:', detailMessage);
    }
    return throwError(() => err);
  })
);
};

```

4. **Register the Pipeline Shapers:** Register `errorInterceptor` in `src/app/app.config.ts` alongside your `credentialsInterceptor`:

```

import { ApplicationConfig, provideHttpClient, withInterceptors,
withXsrfConfiguration } from '@angular/core';
import { credentialsInterceptor } from
'./interceptors/credentials.interceptor';
import { errorInterceptor } from
'./interceptors/error.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([credentialsInterceptor, errorInterceptor]),
      withXsrfConfiguration({
        cookieName: 'XSRF-TOKEN',
        headerName: 'X-XSRF-TOKEN',
      })
    )
  ]
};

```

[NOTE]

Part B: Optimistic UI Deletion with Automatic Snapshot Rollback

Implement optimistic UI updates for course deletion. When Abeba clicks Delete Course, we remove the entity from our `SignalStore` immediately. If the server rejects the call (for instance, if active student enrollments exist), we restore the previous snapshot seamlessly.

1. Open `src/app/store/course.store.ts`.
2. Implement the `deleteCourse` transactional method inside your store's `withMethods` definition:

```

import { inject } from '@angular/core';
import { signalStore, withMethods, patchState } from

```

```

'@ngrx/signals';
import { removeEntity, setAllEntities } from
'@ngrx/signals/entities';
import { catchError, EMPTY } from 'rxjs';
import { CourseService } from '../services/course.service';

export const CourseStore = signalStore(
  { providedIn: 'root' },
  withMethods((store, svc = inject(CourseService)) => ({
    deleteCourse(id: number) {
      // 1. Take a snapshot of current entities BEFORE
      mutating local state variables
      const previousSnapshot = store.entities();

      // 2. Instant visual feedback – remove entity
      immediately from local UI arrays
      patchState(store, removeEntity(id));

      // 3. Dispatch API call to backend server
      svc.delete(id).pipe(
        catchError(err => {
          // 4. Server rejected request – restore previous
          snapshot and set error message
          patchState(store, setAllEntities(previousSnapshot));
          patchState(store, {
            error: 'Cannot delete course: active student
enrollments exist.'
          });
          return EMPTY;
        })
      ).subscribe();
    }
  }));
);

```

[!CAUTION]

Execution Order Rule: The `store.entities()` snapshot **MUST** be captured before calling `patchState(store, removeEntity(id))`. If you snapshot after `patchState`, your rollback snapshot will already be missing the deleted item!

[!NOTE]

Part C: SignalR CORS Verification

Real-time status updates streaming over WebSockets must comply with your server's security filters. Now that your backend enforces a named CORS policy, verify that your SignalR hub allows cross-origin connections explicitly.

1. Open `Program.cs` in your Web API project.
2. Locate your SignalR hub mapping block and ensure `.RequireCors("TmsClient")` is explicitly chained to the endpoint routing definition:

```
app.MapHub<TmsHub>("/hubs/tms").RequireCors("TmsClient");
```

3. Restart your API application runtime (`dotnet run`). SignalR WebSockets will now successfully negotiate cross-origin handshakes from `http://localhost:4200` without connection abort faults.